Anonymous Communication using Onion Routing

Aman Bansal
Syamantak Kumar

Introduction

History - Mix Networks

- Chain of proxy servers makes communication difficult to trace
- Mix Node : Collects and decrypts messages till sufficient number of messages received and then shuffles & forwards

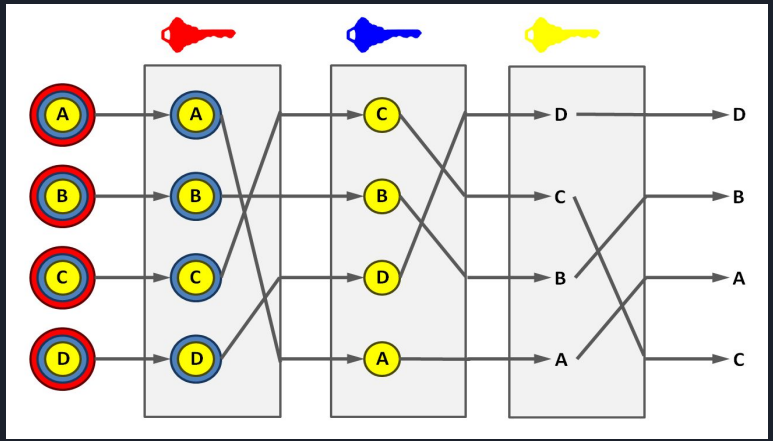
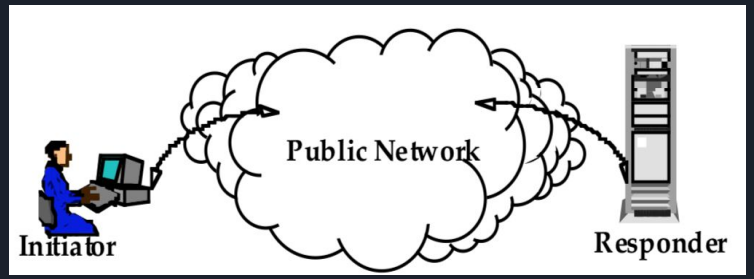


Figure 1: Example of a mix network

Basic Details

- *Onion routing (OR) Network* - Based on the concept of mix networks
  - Consists of specially designed “onion routers” ≡ “mix routers” which are interconnected with using *long-standing* (fixed) connections
  - Different from mix routers as they cannot keep holding messages and use synthetic traffic to overcome that



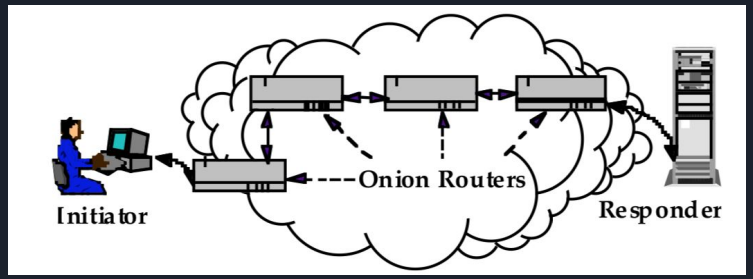


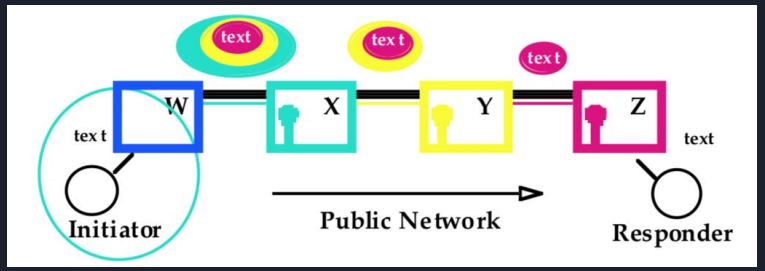
Figure 2: Difference in public network and OR network

Terminology

- Initiator
- Responder
- Forward Direction
- Backward Direction
- Application Proxy
- Onion Proxy
- Onion
- Entry Funnel
- Exit Funnel

Routing Phases

- Connection Setup Phase
  - Initiator’s onion proxy decides sequence of onion routers
  - Each router receives the onion, decrypts and forwards it
  - Completely peeled onion is received by the onion proxy



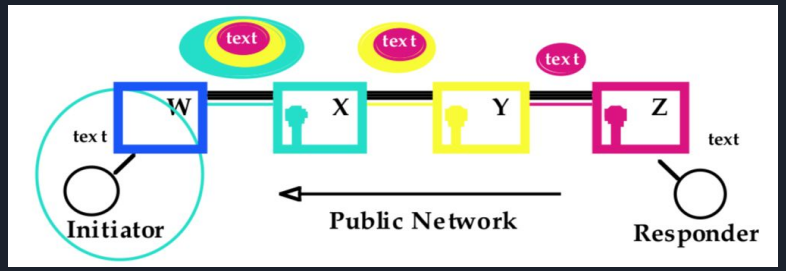


Figure 3: Data Movement in both directions

Routing Phases

- Data Movement Phase
  - Path fixed, every router in path keyed & knows crypting algos
  - Forward Direction - removes encryption layer at each successive router
  - Backward Direction - adds encryption layer at each successive router

- Termination Phase
  - Either end of the connection or any intermediate router can terminate
  - Equivalent to the other side closing the TCP connection

Specifics

Proxies

- Transparent interface for communication between two applications which are otherwise unable to establish direct socket connection to each other

- OR uses 2 types of proxies:
  - Application Proxy
  - Onion Proxy

Application Proxy

- Layer of Abstraction between OR network and the application
- Connection Setup :
  - Decides whether to accept or deny request
  - Connects and sends a *“standard structure”* and the destination address to the onion proxy
  - Waits for an error code before sending the data
- Data Movement : Converts data from application into fixed-size cells
- Termination : Passes relevant error code to or from the application

Onion Proxy

- Waits for request from Application Proxy - decides to serve or not
- Connection Setup :
  - Selects path for reaching destination
  - Builds and Sends the layered onion to the entry funnel
  - Onion establishes the anonymous connection
  - Then sends the standard structure and future data over the network
- Data Movement : Acts as a data relay
- Termination : Application proxy closes the socket with the onion proxy

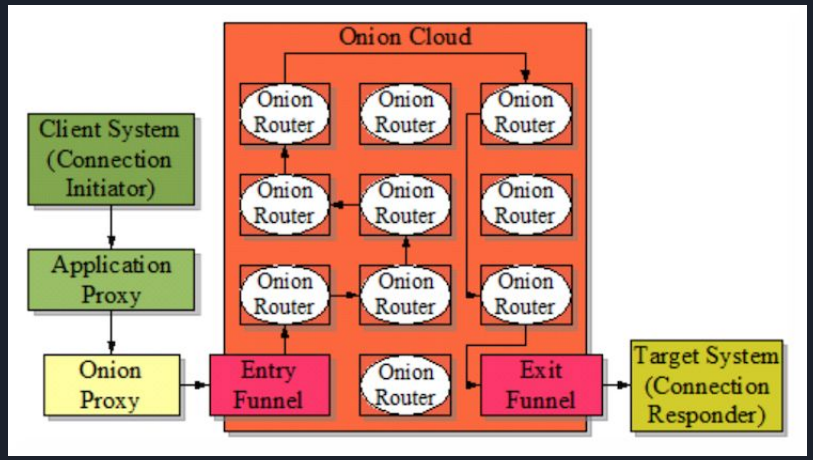
Entry and Exit Funnels

- Entry Funnel
  - Multiplexes connections from various onion proxies to the OR network
  - Any onion proxy first opens a socket connection with entry funnel of the first onion router
  - Sends onion to funnel, which further sends it to the first router

- Exit Funnel
  - Multiplexes connections from OR network to various onion proxies
  - Terminal Router passes data to its exit funnel
  - Tries to establish a connection with dest. Addr, and returns appropriate error code
  - For rest of the data, acts as a relay between onion proxy and last router

Onions

- Multi-layered data structure which encodes the path and other information which is going to be used during the communication
- Each layer encrypted using public key of intended router



Structure of an Onion

- The first bit
- Version
- Key Seed Material:
  - 128-b key_1, key_2, key_3 using SHA
  - First 8 bytes for DES and 16 for RC4
- ‘Back’ field: uses key_2
- ‘Forward’ field: uses key_3
- Destination Address and Port
- Expiration Time

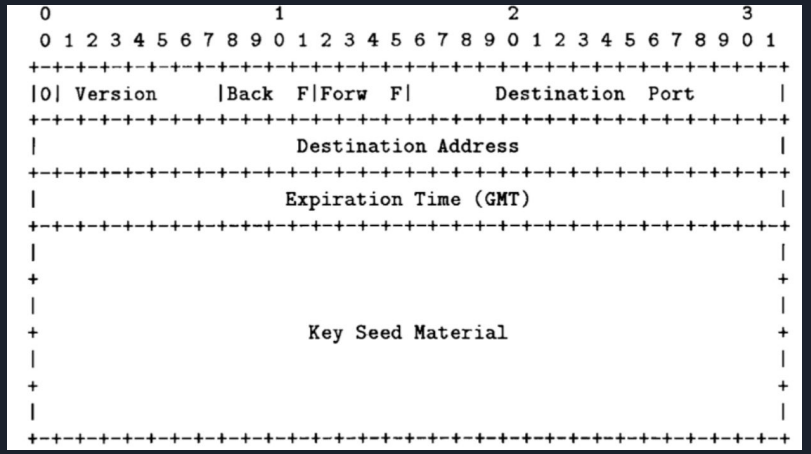


Figure 4: Structure of a layer of an Onion

Construction of an Onion

- Initially the onion consists of 100 Bytes of random data

- For each layer in order from innermost to outermost:
  - Prepend the layer to the onion
  - Encrypt first 128 Bytes of onion using RSA
  - Encrypt the reminder using DES OFB with an IV of 0 and key_1

Onion Router Interconnection

- All connections established and keyed during Network setup
- To open a connection with a neighbour :
  - Connection Setup
    - The initiating onion router opens a socket to the neighboring router
  - Keying
    - STS(Secure Token Service) used to get 2 DES 56-bit keys.
    - Link Encryption - uses DES OFB encryption with the above keys
  - After successful keying, data divided into fixed-sized cells

Cells

- Transmission of data takes place at the granularity of cells.
- Its fields are as follows:
  - ***ACI***
  - ***Command***
  - ***Length***
  - ***Payload***

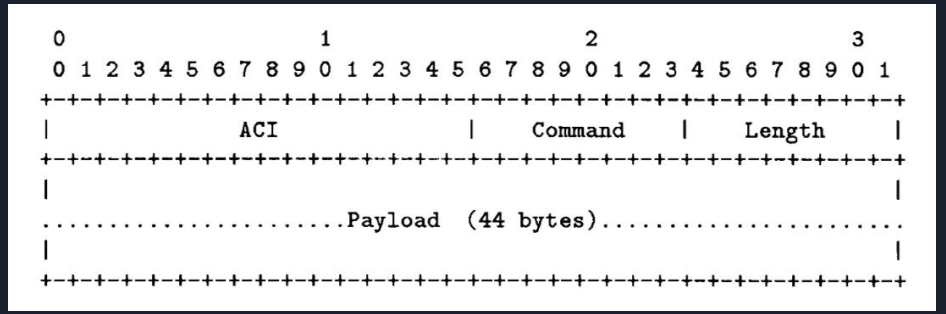


Figure 5: Structure of a cell

Types of Cells

- CREATE :
  - Length - Link Encrypted, Payload - Already Encrypted
  - Chooses a new ACI for the link and stores mapping
  - Higher (Lower) IP/port maps top-half (bottom-half) of the address space

- DATA :
  - Length and Payload - Crypted using cryptographic functions defined at setup
  - Forward Direction - Length and Payload repeatedly encrypted by onion proxy using router specific functions and decrypted at each router
  - Backward Direction - exact reverse happens

Types of Cells

- DESTROY :
  - Length & Payload : Link Encrypted, sent upon connection termination
  - ACI field refers to the broken connection
  - Each OR sends ACK on receiving DESTROY cell
  - Mappings can be removed upon successful receipt of ACK

- PADDING :
  - Used to inject data to further confuse traffic analysis
  - Dropped upon receipt

Threat Model

Security Goals

- ***Sender Activity*** : Knowledge that the sender has sent something

  Receiver Activity : Knowledge that the receiver has received something

- ***Sender Content*** : Knowledge that the sender sent a particular content

  Receiver Content : Knowledge that the receiver received a particular content

- ***Source-destination Linking*** : The knowledge that a particular sender is sending something to a particular receiver.

Adversary Model

1. ***Observer***
2. ***Disrupter***
3. ***Hostile User***
4. ***Compromised Core Onion Router (COR)***

Note that proving the security of the network w.r.t. the adversaries which are composed of one or more CORs is sufficient for proving the security of the network

Adversary Model

We further categorize the class of compromised CORs adversaries :

- ***Single Adversary***
- ***Multiple Adversary***
- ***Roving Adversary***
- ***Global Adversary***

Note that OR doesn’t provide any security guarantee against the global adversary. Therefore, it is sufficient to consider only the roving adversary.

Possible Side-Channel Attacks

- ***Marker Attack*** : A marker is basically a data which upon being sent generates a observable pattern in the encrypted traffic. Can be used to narrow down the set of next hops.

- ***Timing attack***: Each compromised router tracks the data rate of a particular session (*timing signature*). Can be used to identify nodes belonging to the same connection.

Security Analysis

Assumptions & Notation

Assumptions:

1. Adversary characterised by ‘c’ - Number of compromised routers
2. Path from sender to receiver a random walk (No cycles of length 1)
3. CORs affected in a previous round, which are not now, are assumed to be healed instantly

Notations :

1. C_i denotes the set of CORs which are compromised in the i^{th} round.
2. ‘r’ denotes the total number of CORs in our network
3. ‘n’ is the (variable) length of the route $R = \{R_1, R_2, ..., R_n\}$.

We will do security analysis in 2 configurations : Remote-COR and Local-COR.

Remote-COR Configuration

The user has secure remote access to the first COR in the route. For the i^{th} round:

1. $C_i \not\ni R_1$ and $C_i \not\ni R_n$: The adversary learns nothing.
2. $C_i \ni R_1$: Only Sender Activity compromised. $P(C_i \ni R_1) = c/r$.
3. $C_i \ni R_n$: Receiver Activity and Content compromised. $P(C_i \ni R_n) = c/r$.
4. $C_i \ni R_1$ and $C_i \ni R_n$: Sender Activity, Receiver Activity, Receiver Content compromised. $P(C_i \ni R_1 \text{ and } C_i \ni R_n) = c^2/r^2$.

Remote-COR Configuration

Therefore, the goal of the adversary is to compromise the first or the last router.

- At route-setup time, the probability that at least one COR in the route of length n is present in C_i is given by

$$1 - P(R \cap C_1 = \phi) = 1 - (r - c)^n / r^n$$

- If the adversary compromises a node in the route, it can, through timing analysis , ultimately reach R_1 and R_n in linear time.

Local-COR Configuration

- The user owns a COR and uses it as the first COR.

- Therefore the first and last CORs are always protected by the integrity of the users and the adversary can not compromise any security goals.

Thank you !

References

[1] Michael G. Reed, Paul F. Syverson, and David M. Goldschlag. Anonymous connections and onion routing.

[2] Paul F. Syverson, Gene Tsudik, Michael G. Reed, and Carl E. Landwehr. Towards an analysis of onion routing security.

[3] David Chaum. Untraceable electronic mail, return addresses and digital pseudonyms.

[4] Michael G. Reed, Paul F. Syverson, and David M. Goldschlag. Proxies for anonymous routing.

[5] Alfred Menezes, Paul C. van Oorschot, and Scott A. Vanstone. Handbook of Applied Cryptography.

[6] Michael K. Reiter and Aviel D. Rubin. Crowds: Anonymity for web transactions.

[7] Whitfield Diffie, Paul C. van Oorschot, and Michael J. Wiener. Authentication and authenticated key exchanges.

[8] Daniel Arp, Fabian Yamaguchi, and Konrad Rieck. Torben: A practical side-channel attack for deanonymizing tor communication.